

JAVA INTERVIEW PREPARATION

CHAPTER 21

DATA STRUCTURES AND COMPLEXITY

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Practical Data Structures and Complexity in Java: Choosing Shapes That Match the Workload

Senior / Lead Java Developer Reading Chapter

Data-structure interviews are not tests of memorized Big-O tables alone. At senior level, the interviewer expects a candidate to connect access patterns, memory representation, concurrency, domain invariants, and operational scale. The right structure makes the common operation cheap and the invalid state difficult to represent.

Begin with Operations, Not Class Names

Before selecting a collection, write down the dominant operations and constraints:

- Lookup by identifier or by position?
- Preserve insertion order, sorted order, or no order?
- Insert at one end, both ends, or arbitrary positions?
- Retrieve the smallest or highest-priority item repeatedly?
- Need range queries, prefix queries, or graph traversal?
- What is the maximum cardinality and memory budget?
- Is mutation single-threaded, externally locked, or concurrent?

workload question	likely starting point
lookup by unique key	HashMap
membership test	HashSet
indexed iteration	ArrayList
FIFO or double-ended operations	ArrayDeque
repeated minimum/maximum	PriorityQueue
sorted keys and range navigation	TreeMap / TreeSet
enum flags	EnumSet

This is a starting point, not a replacement for measurement or domain analysis.

Complexity Is a Growth Model

Big-O describes how cost grows as input grows, usually ignoring constants and hardware effects.

$O(1)$	cost does not grow with n in the model
$O(\log n)$	problem shrinks by a constant factor each step
$O(n)$	work grows proportionally with elements
$O(n \log n)$	efficient comparison sorting and divide-and-conquer pattern
$O(n^2)$	pairwise or nested full scans

Expected $O(1)$ hash lookup can lose to $O(\log n)$ tree lookup for tiny collections because of constants. An $O(n)$ array scan can be faster than pointer-heavy traversal due to locality. Complexity predicts scaling; benchmarking and profiling quantify a real implementation.

Arrays and Dynamic Arrays

An array provides fixed-size contiguous storage and $O(1)$ indexed access. Primitive arrays pack values efficiently. Object arrays store references.

```
int[] latencyBuckets = new int[100];
latencyBuckets[42]++;
```

ArrayList adds dynamic size and collection APIs around an Object array. It is usually the best general list, but primitive boxing and middle shifts remain relevant. Arrays are useful for fixed dimensions, performance-sensitive primitives, lookup tables, heaps, and adjacency structures when the index space is dense.

Stacks, Queues, and Deques

ArrayDeque implements a resizable circular buffer and efficiently adds or removes at both ends.

```
backing circular array
  head -> [B][C][D][_][_][A] <- tail wraps
logical order: A, B, C, D
```

Use it as a stack:

```
Deque<Node> stack = new ArrayDeque<>();
stack.push(root);
Node next = stack.pop();
```

Or as a FIFO queue:

```
Deque<Job> queue = new ArrayDeque<>();
queue.addLast(job);
Job next = queue.removeFirst();
```

Prefer Deque to the legacy Stack class. ArrayDeque rejects null, which keeps methods such as poll returning null unambiguous. It is not thread-safe; use BlockingQueue or ConcurrentLinkedDeque when the concurrency contract requires them.

Priority Queues and Heaps

PriorityQueue is normally a binary heap stored in an array. It provides $O(1)$ access to the head, $O(\log n)$ insertion, and $O(\log n)$ removal of the head.

heap logical tree	array representation
<pre> 2 / \ 5 4 / \ / \ 9 8 7</pre>	<pre>[2, 5, 4, 9, 8, 7]</pre>

```
PriorityQueue<Job> ready = new PriorityQueue<>(
    Comparator.comparing(Job::deadline)
                .thenComparing(Job::id)
);
ready.add(job);
Job mostUrgent = ready.poll();
```

Iteration is not sorted. Only repeated poll returns priority order. Removing an arbitrary element is generally linear because the heap does not maintain a key-to-index lookup.

PriorityQueue is not a scheduler by itself. It does not wait until a future deadline, provide persistence, or coordinate multiple service instances. Java's DelayQueue or a durable scheduling system may match those requirements better.

Hash Tables and Trees

HashMap offers expected $O(1)$ key lookup without ordering. TreeMap uses a balanced search tree with $O(\log n)$ lookup and sorted navigation.

```
NavigableMap<Instant, Event> timeline = new TreeMap<>();
Map.Entry<Instant, Event> next = timeline.ceilingEntry(now);
NavigableMap<Instant, Event> hour =
    timeline.subMap(start, true, end, false);
```

Choose a tree when sorted iteration, nearest-neighbor operations, or ranges are primary. Do not sort an entire HashMap on every request if a continuously sorted index is what the application needs. Conversely, do not pay tree costs only to perform exact key lookup.

Building an LRU Cache

LinkedHashMap can maintain access order and evict the eldest entry.

```
final class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int maximumSize;

    LruCache(int maximumSize) {
        super(16, 0.75f, true); // access order
        this.maximumSize = maximumSize;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > maximumSize;
    }
}
```

This illustrates combining a hash table for lookup with a linked order for eviction. It is not automatically thread-safe and lacks production cache features such as time expiration, weight limits, refresh, statistics, asynchronous loading, and stampede control. Prefer a mature cache library for serious caching, but understand the underlying composition.

Graph Representation

A graph models entities connected by edges. An adjacency list is usually efficient for sparse graphs.

```
Map<String, List<String>> adjacency = new HashMap<>();
adjacency.computeIfAbsent("A", ignored -> new ArrayList<>()).add("B");
adjacency.computeIfAbsent("A", ignored -> new ArrayList<>()).add("C");
```

```
A -> B, C
B -> D
C -> D
D ->
```

An adjacency matrix uses $O(V^2)$ space but offers constant-time edge checks and can be appropriate for small dense graphs. Representation should follow graph density and operations.

Breadth-First Search

Breadth-first search uses a queue and visits nodes by distance in an unweighted graph. A visited set prevents cycles and repeated work.

```

List<String> breadthFirst(Map<String, List<String>> graph, String start) {
    List<String> order = new ArrayList<>();
    Set<String> visited = new HashSet<>();
    Deque<String> queue = new ArrayDeque<>();

    visited.add(start);
    queue.addLast(start);

    while (!queue.isEmpty()) {
        String node = queue.removeFirst();
        order.add(node);

        for (String neighbour : graph.getOrDefault(node, List.of())) {
            if (visited.add(neighbour)) {
                queue.addLast(neighbour);
            }
        }
    }
    return order;
}

```

With adjacency lists, time is $O(V + E)$ because each vertex and edge is processed a bounded number of times. Space is $O(V)$ beyond the graph for queue, visited set, and output.

Mark nodes visited when enqueueing, not when dequeuing, to avoid adding the same node many times through different incoming edges.

Depth-First Search and Recursion Risk

Depth-first search uses a stack, explicit or recursive. It supports reachability, cycle detection, topological algorithms, and component discovery.

Recursive code is concise, but a very deep graph can overflow the Java thread stack. An explicit ArrayDeque makes depth a heap-backed data-structure concern and gives more control.

```

Deque<String> stack = new ArrayDeque<>();
stack.push(start);
while (!stack.isEmpty()) {
    String node = stack.pop();
    if (!visited.add(node)) continue;
    for (String neighbour : graph.getOrDefault(node, List.of())) {
        stack.push(neighbour);
    }
}

```

Traversal order depends on neighbor order and stack insertion order. Do not promise deterministic output unless representation and algorithm deliberately provide it.

Tries and Prefix Search

A trie stores keys by successive characters or tokens. Lookup is $O(L)$ in key length rather than the number of stored keys, and prefix traversal is natural.

```

root
  c -> a -> t
      \ -> r
  d -> o -> g

```

Tries can consume substantial memory because each node has child structure and metadata. Compressed tries, sorted arrays, database indexes, or search engines may be better. Use them when prefix behavior and scale justify the representation.

Union-Find for Connectivity

Disjoint-set union tracks connected components with parent links. Path compression and union by rank or size make operations nearly constant amortized time.

```
int find(int x) {
    if (parent[x] != x) {
        parent[x] = find(parent[x]);
    }
    return parent[x];
}
```

It is effective for network connectivity, clustering, and Kruskal's minimum-spanning-tree algorithm. It answers connectivity under unions; it is not a general replacement for graph storage and does not naturally handle edge deletion.

Boundedness and Backpressure

Data structures also encode capacity policy. An unbounded queue accepts work faster than consumers can process it, converting overload into memory growth and latency.

```
producer rate 5,000/s -> unbounded queue -> consumer rate 3,000/s
backlog grows 2,000/s until memory or latency fails
```

A bounded `BlockingQueue` can reject, block, or trigger load shedding according to the executor policy. The bound should come from latency and capacity goals, not an arbitrary large number.

Similarly, every cache needs a size or weight policy, every deduplication set needs a retention window, and every batch needs a maximum cardinality. Boundedness is part of correctness in production.

Complexity Traps in Application Code

Nested use of linear collections can create accidental quadratic behavior:

```
for (Order order : orders) {
    if (blockedCustomerIdsList.contains(order.customerId())) {
        reject(order);
    }
}
```

If both collections contain n elements, repeated `List.contains` can approach $O(n^2)$. Converting the membership side to a `HashSet` changes expected lookup to $O(1)$, making the loop expected $O(n)$, at the cost of building and storing the set.

Other traps include sorting repeatedly inside a loop, concatenating immutable strings in large loops, removing from the front of `ArrayList`, and recursively traversing an unbounded depth.

Production and Senior-Level Reasoning

Explain a choice with the full chain: operation profile, expected complexity, memory representation, ordering semantics, concurrency boundary, and failure behavior. For example: "I chose a bounded `ArrayBlockingQueue` because producers can exceed consumers; the bound caps retained work, and rejection activates load shedding before the JVM exhausts memory."

Monitor cardinality, queue depth and age, cache hit rate and eviction, distribution of key popularity, allocation rate, retained size, and tail latency. A theoretically suitable structure can still fail if the workload assumptions are wrong.

Interview-Ready Summary

Choose structures from dominant operations and invariants. Arrays and ArrayList serve indexed and sequential access; ArrayDeque serves stacks and queues; PriorityQueue serves repeated best-item retrieval; hash tables serve exact lookup; trees serve sorted and range operations; adjacency lists serve sparse graphs.

Complexity is a scaling model, not a stopwatch. Consider constants, locality, boxing, allocation, cardinality, and concurrency. Encode boundedness explicitly for queues, caches, sets, and batches. Explain alternatives and production failure modes rather than naming one collection in isolation.

Revision Notes

- Start with operations, ordering, cardinality, memory, and concurrency requirements.
- Big-O describes growth, not exact runtime.
- Arrays offer fixed contiguous storage and $O(1)$ indexing.
- ArrayList is the normal dynamic indexed list.
- ArrayDeque is preferred for stacks and double-ended queues.
- PriorityQueue is a heap; iteration is not sorted.
- Heap insertion and head removal are $O(\log n)$.
- HashMap favors exact lookup; TreeMap favors ordering and ranges.
- LinkedHashMap can combine lookup with insertion or access order.
- A simple LRU is not automatically a production cache.
- Adjacency lists suit sparse graphs.
- BFS uses a queue and visits an unweighted graph in $O(V + E)$.
- Mark BFS nodes visited when enqueueing.
- DFS recursion can overflow on deep graphs.
- Tries support prefix lookup but can use substantial memory.
- Union-find efficiently tracks connectivity under union operations.
- Unbounded queues turn overload into memory retention and latency.
- Repeated List.contains inside a loop can create quadratic behavior.
- Sets can convert repeated membership checks to expected constant time.
- A senior answer connects complexity to memory, concurrency, and operations.
- Validate assumptions with production cardinality and profiling.